OST
Ostschweizer
Fachhochschule

Lab 13

FTP MachLe MSE
FS 2026

Reinforcement Learning (solutions)

MSE MachLe
WÜRC

Lernziele/Kompetenzen

- You know the definition of a finite *Markov decision process* (MDP) and the definition of a *Markov Reward Process* (MRP).
- You understand the two basic algorithms for iteratively approximating the dynamic programming task for finite MDPs in the special cases of one-step, tabular, modelfree TD (time-difference) methods, namely *value iteration* and *Q-learning*.
- You know the difference between *on-policy* and *off-policy* learning.
- You can explain the difference between *value iteration* and *policy iteration*.
- You know the trade-off between *exploitation* and *exploration*.

---

1. General Questions: Reinforcement Learning [A, II]

a) We can try to think about it in terms of the limitations a finite MDP imposes in a problem definition and whether any approximations could exist within the framework.

(i). *The Markov property*, if not only the previous state and action selected influence which will be the next state. An example of this could be a modified game of chess where the order in which the pieces were moved affects the possible moves and the sequence information wasn't available (or only partially available) at the time of making a decision. Information from the past that can affect the next state would be missing, breaking the Markov property.

(ii). *The action and state sets must be finite*. Any problem with infinite available actions or states would need alternative representations, such as grouping them into subsets and use these sets.

(iii). *Rewards must be numerical*. If the rewards the environment gives back are not numerical we would need to encode them as numbers. This can be a highly difficult task as the rewards may not translate naturally to numbers. For example, if the rewards were your family's verbal feedback on how good the meal you prepared was, it would be difficult to convert it into a number and capture correctly its intensity.

b) This is likely an exploration issue where the agent is unable to find the exit the first time and therefore doesn't know there's anything better than 0 as a reward. Potential solutions include having each non-goal state be worth -1, and/or extending the episode length. This

would mean states the agents visits a lot (particularly around the start) will get worse and worse values so it will want to move away from there and eventually find the goal (essentially reaching the goal stops it being in pain).

c) The code could look like this. Note the inverse indexing.

```
r = [-1, 2, 6, 3, 2]
gamma = 0.5

for g_i in range(len(r)):
  ret = sum([gamma**i * r_i for i, r_i in enumerate(r[g_i:])])
  print("G_"+ str(g_i) + " = " + str(ret))

G_0 = 2.0
G_1 = 6.0
G_2 = 8.0
G_3 = 4.0
G_4 = 2.0
```

d) The discounted cumulative reward can be caluclated as a sum of a geometric sequence using the formula for $|q| < 1$:

$$\sum_{k=0}^{\infty} q^k = \frac{1}{1-q}$$

$$G_0 = 2 + \gamma\ 7 + \gamma^2\ 7 + \cdots = 2 + \gamma\ (\sum_{k=0}^{\infty} \gamma^k\ 7)$$

$$= 2 + \gamma\ 7\ (\sum_{k=0}^{\infty} \gamma^k) = 2 + \gamma\ 7\ (\frac{1}{1-\gamma}) = 2 + 0.9 \times 7 \times 10 = \underline{\underline{65}}$$

$$G_1 = 7 + \gamma\ 7 + \gamma^2\ 7 + \cdots = 7 \cdot (\sum_{k=0}^{\infty} \gamma^k) = 7 \cdot \frac{1}{1-\gamma} = \underline{\underline{70}}$$

e) In general it would play worse. The chance that the correct action for a situation in the long run is the first one that returns a positive reward is pretty small, particularly if there are a large number of actions available. It would also be unable to adapt to opponents that slowly altered behaviour over time.

2. SARSA Reinfocrement Learning [A,II]

The Jupyter notebook solution can be found on moodle:
`SARSA_TaxiDriver.jpynb`.

3. Q-Learning on the Frozen Lake Environment [A,II]

The Jupyter notebook solution can be found on moodle:
`QLearning_FrozenLake.jpynb`.